

Apuntes de Python

Apuntes introductorios a la programación procedimental en Python

Federico Melo Barrero
f.melo@uniandes.edu.co

Copyright © 2020, 2021 Federico Melo Barrero.

Todos los derechos reservados.

Ninguna parte de este documento, incluyendo el diseño de la portada, figuras e imágenes, puede ser reproducida, registrada, almacenada o transmitida en manera alguna ni por ningún medio, ya sea electrónico, eléctrico, fotoquímico, mecánico, electroóptico, magnético, de grabación, de fotocopia o cualquier otro sin permiso previo y por escrito del autor.

Índice

1. Programar	3
I Fundamentos	4
2. Valores, variables y expresiones	4
2.1. Valores	4
2.2. Variables	4
2.2.1. Sobreescritura y tipado dinámico	5
2.3. Expresiones	5
2.3.1. Operadores aritméticos	5
2.3.2. Operadores de cadenas de caracteres	6
2.3.3. Asignaciones con operador	6
3. Funciones	6
3.1. Funciones de entrada y salida	7
3.1.1. Función print—	7
3.1.2. Función input—	7
II Condicionales	8
4. Booleanos	8
4.1. Operadores relacionales	8
4.1.1. Operadores de identidad	8
4.2. Operadores lógicos	8
4.2.1. Tablas de verdad	8
5. Estructura de control condicional	8
5.1. Cascadas de condicionales vs condicionales en secuencia	8
5.2. Expresión ternaria	8
6. Abordar problemas de programación con condicionales	8
7. Diccionarios	8
III Ciclos	9
8. Ciclo while	9
9. Ciclo for	10
10. ¿while o for?	10
10.1. De for a while	10

Preámbulo

Tomé estos apuntes cuando cursé Introducción a la Programación en la Universidad de los Andes, entre agosto y noviembre de 2020. Fueron refinados en mi tiempo como monitor en 2021, luego como tutor en el centro de enseñanza de programación de la universidad en 2022, y tras eso como asistente de investigación en ese mismo centro desde el 2022 hasta el 2024. Finalmente, fueron revisados en mi etapa como profesor de la asignatura entre el 2025 y 2026, pero no fueron modificados sustancialmente¹: siguen siendo apuntes de un estudiante para estudiantes.

1. Programar

Programar es, en esencia, escribir instrucciones que puedan ser ejecutadas por una máquina.

En general, escribir instrucciones para una máquina es más difícil que escribirlas para un humano por (al menos) tres razones:

- La máquina no entiende español, por lo que las instrucciones deben estar escritas en un lenguaje diseñado para que la máquina entienda. Existen centenas de dichos **lenguajes de programación**. **Python** es uno de ellos.
- Los lenguajes naturales (como el español) tienen reglas que determinan cómo organizar las palabras para que esté correctamente escrito. A eso se le llama **sintaxis**. Es fácil para un humano entender español ligeramente mal escrito, pero una máquina no puede interpretar Python con sintaxis incorrecta.
- El significado de un conjunto de palabras es su **semántica**. Los humanos pueden entender la intención detrás de instrucciones mal formuladas, pero una máquina ejecuta lo escrito de forma literal, así tenga poco sentido o no haga lo que se pretendía.

Esos tres factores hacen que programar sea difícil y que sea ampliamente considerado un arte.



Tip. En estos apuntes se usa mucho la palabra sintaxis. En esencia, es el orden en que se escriben las cosas, que es muy importante al programar.

¹De hecho, los apuntes están presentados en el mismo formato en el que originalmente fueron escritos; la clase de L^AT_EX que determina su estilo la escribí en noviembre del 2020.

Parte I

Fundamentos

2. Valores, variables y expresiones

2.1. Valores

Los textos en español se componen de palabras; los programas en Python se componen de **valores**. Ejemplos de valores son números y secuencias de caracteres. Algunos valores:

```
42
3.14
"Hola"
```

El primer valor es el número entero 42, el segundo es el número decimal 3.14 y el tercero es una secuencia de cuatro caracteres del teclado: H, o, l, a.

Análogo a como las palabras tienen categorías: sustantivos, verbos, etc.; los valores tienen **tipos de dato**²:

Tipo de dato	Nombre (inglés)	Traducción	Descripción	E.g.
<code>int</code>	<i>integer</i>	número entero	Números enteros	42, -17, 0, 1_000_000
<code>float</code>	<i>floating point number</i>	número de punto flotante	Números decimales	3.14, -0.5, .35
<code>str</code>	<i>string</i>	cadena de caracteres ³	Secuencias de caracteres del teclado	"Hola", 'u78%6#a'

Cuadro 1: Algunos tipos de dato primitivos en Python

Las cadenas de caracteres deben escribirse entre comillas para que la máquina entienda dónde comienza y dónde termina la secuencia de caracteres que se está representando. Hay cuatro formas de escribirlas:

- Usando comillas dobles: `"Hola"`
- Usando comillas simples: `'H014'`
- Usando triples comillas dobles: `"""Hola"""`
- Usando triples comillas simples: `'''Python'''`

Naturalmente, si entre los caracteres de una cadena se quiere incluir algún tipo de comillas, es sensato delimitar la cadena con otro tipo de comillas, para que no haya ambigüedad. E.g.: `"Don't!"`, representa la secuencia de caracteres de teclado D, o, n, ', t, !. Si se hubiese escrito `'Don't!'`, no es claro cuál comilla termina la cadena.

Las triples comillas (dobles o simples) permiten representar cadenas de caracteres que contienen saltos de línea.

```
"""Hola
Mundo"""
```

2.2. Variables

Una **variable** es un nombre que se asigna a un valor. Las variables se utilizan para almacenar valores en la memoria del programa.

Para definir una variable se utiliza el **operador de asignación**, que es el carácter =.

²Hay más tipos de dato en Python, que se exponen más adelante.



Tip. El símbolo de igualdad en matemáticas y el operador de asignación en Python utilizan el mismo carácter: `=`. Sin embargo, en cada contexto significa cosas diferentes.

En programación sucede bastante que se utilizan caracteres que en algún otro contexto significan otra cosa.

La sintaxis, de izquierda a derecha, se compone de: el nombre de la variable, el operador de asignación y el valor que se le asigna a la variable. E.g.:

```
persona = "David"
altura = 1.77
```

Allí:

- Se toma la secuencia de caracteres `D`, `a`, `v`, `i`, `d` y se almacena en la memoria del programa bajo el identificador `persona`.
- Se toma el número decimal `1.77` y se almacena en la variable `altura`.

Tiene sentido otorgar nombres a los valores para poder usarlos después. Los identificadores de las variables deben seguir las siguientes reglas:

- Deben ser alfanuméricos: letras, números y guiones bajos.
- No pueden contener espacios; las palabras dentro del identificador se separan con guiones bajos: `nombre_de_la_variable`.
- No pueden empezar por dígitos y no deben empezar por mayúsculas.
- No pueden ser **palabras reservadas** del lenguaje, que son palabras específicas en inglés que ya tienen un significado en Python.

2.2.1. Sobreescritura y tipado dinámico

Se puede sobrecribir el valor de una variable por otro y la máquina olvida el valor anterior. E.g.:

```
persona = 3
```

Ahora, la variable `persona` tiene el valor `3` y deja de hacer referencia el valor anterior, `"David"`.

En el ejemplo anterior, la variable se definió con un valor de tipo `str` y luego se cambió a un valor de tipo `int`. Cambiar el tipo de dato de una variable es posible en Python, pero no lo es en muchos otros lenguajes de programación. A esa característica de Python se le llama **tipado dinámico**⁴.

2.3. Expresiones

Una **expresión** es una combinación de valores y operadores que la máquina puede evaluar para obtener un valor. En matemáticas, se puede evaluar la expresión $1 + 1$ para obtener el valor `2`; en Python, es similar.

Los **operadores** son símbolos que representan operaciones entre valores de un tipo de dato determinado.

2.3.1. Operadores aritméticos

Las operaciones que se pueden realizar con valores `int` y `float` son las operaciones aritméticas convencionales que se pueden realizar con números:

⁴Lo descrito en este párrafo es realmente una consecuencia del tipado dinámico, no una definición. Una definición escapa al alcance de estos apuntes, pero podría ser: un lenguaje es dinámicamente tipado si su sistema de tipos realiza verificaciones en tiempo de ejecución, en lugar de en tiempo de compilación.

Operador	Símbolo	Matemáticas	Python	Resultado
Exponenciación	**	3^4	<code>3**4</code>	81
Multiplicación	*	7×6	<code>7 * 6</code>	42
División	/	$15/4$	<code>15 / 4</code>	3.75
División entera	//	$\lfloor 15/4 \rfloor$	<code>15 // 4</code>	3
Módulo	%	$10 \bmod 3$	<code>10 % 3</code>	1
Suma	+	$10 + 7$	<code>10 + 7</code>	17
Resta	-	$20 - 8$	<code>20 - 8</code>	12

Cuadro 2: Operadores aritméticos en Python

Las operaciones se pueden combinar para construir expresiones más complejas. E.g.: $2 + 4 * 2$ evalúa a 10. Python no evalúa ingenuamente de izquierda a derecha, sino que respeta la **jerarquía de operaciones**⁵ clásica de la aritmética.

División entera y módulo. Seguramente el único par de operadores no familiares en la tabla 2 son los operadores de división entera (`//`) y módulo (`%`).

La **división entera** consiste en tomar la parte entera del resultado de una división, es decir, de descartar los decimales (si los hay). Eso no es lo mismo que aproximar el resultado al entero más cercano. E.g., la división (convencional) $15 / 4$ evalúa a 3.75 mientras que la división entera $15 // 4$ evalúa a 3.

El **módulo** representa la cantidad entera que sobra después de dividir un número entre otro. Matemáticamente, es el residuo de la división. Intuitivamente: si se reparte una torta de 15 pedazos entre 4 personas por igual (lo cual se puede modelar con la división entera $15 // 4$), cada persona recibe 3 pedazos, pero además sobran 3 pedazos que no le tocan a nadie. El módulo es el 3 que sobra; es decir, $15 \% 4$ evalúa a 3.

Usando fórmulas, las partes de la división son:

$$\begin{aligned}\text{dividendo} / \text{divisor} &= \text{cociente con decimales} \\ \text{dividendo} &= \text{divisor} \times \text{cociente entero} + \text{residuo}\end{aligned}$$

Con eso, $\text{dividendo} // \text{divisor} = \text{cociente entero}$ y $\text{dividendo} \% \text{divisor} = \text{residuo}$.

2.3.2. Operadores de cadenas de caracteres

Las operaciones que se pueden realizar con valores `str` son la concatenación y la repetición:

Operador	Símbolo	Descripción	Ejemplo	Resultado
Concatenación	+	Une dos cadenas de caracteres	<code>"Hola" + "Mundo"</code>	<code>"HolaMundo"</code>
Repetición	*	Opera un <code>str</code> con un <code>int</code> ; repite el <code>str</code> tantas veces como indica el <code>int</code>	<code>"Hola" * 3</code>	<code>"HolaHolaHola"</code>

Cuadro 3: Operadores de cadenas de caracteres en Python

2.3.3. Asignaciones con operador

3. Funciones

Similar a como las variables son nombres que se asignan a valores, las funciones son nombres que se asignan a bloques de código. Esto permite escribir un pedazo de código sola una vez y reutilizarlo múltiples veces.

⁵Estos apuntes presuponen familiaridad con la jerarquía de operaciones; sin embargo, en esencia, el orden es: 1) exponenciación; 2) multiplicación y división (incluyendo división entera y módulo); y 3) suma y resta. Las operaciones con igual prioridad se evalúan de izquierda a derecha. Si existen paréntesis (o corchetes), las operaciones dentro de ellos tienen prioridad sobre las externas, y los paréntesis más internos se evalúan antes que los más externos.

Crear una función es entonces asignar un nombre a un bloque de código. Ese bloque de código puede recibir valores que se le pasan a la función para que use en su código.

Antes de crear funciones, vale la pena conocer algunas funciones **integradas**⁶, que son funciones que creó alguien más y que ya vienen predefinidas en Python. A la acción de utilizar una función se le denomina **invocarla** o **llamarla**.

3.1. Funciones de entrada y salida

Las funciones de entrada y salida se utilizan en el programa para interactuar con el usuario.

3.1.1. Función `print`

La función `print` (que traduce literalmente del inglés como imprimir) recibe un argumento y lo muestra en la consola.

3.1.2. Función `input`

La función `input` se utiliza para leer un valor de la consola. Es decir, se utiliza para que el usuario ingrese un valor y se lo asigne a una variable.

Ejercicios:

⁶El término original para esto es *built-in function*, que traduce más naturalmente a función incorporada. En cualquier caso, es una función que viene pre-escrita en el lenguaje.

Parte II

Condicionales

4. Booleanos

4.1. Operadores relacionales

4.1.1. Operadores de identidad

4.2. Operadores lógicos

4.2.1. Tablas de verdad

5. Estructura de control condicional

La estructura de control condicional en Python utiliza la palabra `if`, que traduce al español como *si*. Su sintaxis es: `if valor de tipo bool:`

`bloque de código` Por ejemplo:

```
if True:
    print("Hello, World!")
```

El bloque de código se ejecuta si la condición es verdadera.

5.1. Cascadas de condicionales vs condicionales en secuencia

5.2. Expresión ternaria

6. Abordar problemas de programación con condicionales

7. Diccionarios

Parte III

Ciclos

Python tiene dos tipos de ciclos: `while` y `for`. Un **ciclo** es una estructura de control que permite repetir un bloque de código varias veces.

8. Ciclo `while`

El ciclo `while` ejecuta un bloque de código mientras una condición sea verdadera. Su sintaxis es análoga a la sintaxis del `if`:

```
while valor de tipo bool :
    bloque de código
```

La diferencia es: en el `if`, el bloque de código se ejecuta cuando la condición es verdadera y se ejecuta una sola vez; en el `while`, el bloque de código se ejecuta una y otra vez mientras la condición siga siendo verdadera.

Dicho de otra forma, el bloque de código solo se deja de ejecutar cuando la condición sea falsa. Eso implica que diseñar un ciclo `while` consiste básicamente de:

- Una condición que inicialmente sea verdadera (si no, el ciclo nunca ejecutará).
- El bloque de código que se ejecutará repetidamente.
 - Dentro del bloque de código, debe haber algo que permita que el valor de la condición cambie. De lo contrario, el bloque de código se seguirá ejecutando indefinidamente. A ese tipo de error se le llama un *ciclo infinito*.

Un ejemplo de ciclo `while` es el siguiente:

```
i = 0
while i < 3:
    print(i)
    i += 1
```

En ese ejemplo:

- La condición es `i < 3`, que es verdadera inicialmente.
- El bloque de código imprime el valor de `i` y luego incrementa el valor de `i` en 1.
 - Como siempre se incrementa el valor de `i`, eventualmente `i` valdrá 3 y la condición será falsa. En ese punto, el bloque de código se deja de ejecutar.

Con lo anterior en mente, resulta evidente que el ciclo se ejecuta 3 veces:

1. La primera ejecución se debe a que `i` vale 0 y 0 es menor que 3. Se imprime el valor 0 y luego se incrementa `i` en 1. Al final de esa ejecución, `i` vale 1.
2. Luego, se revisa si se cumple la condición. En este caso, sí se cumple, porque 1 es menor que 3. Se imprime 1, y se incrementa, de forma que `i` ahora vale 2.
3. En la tercera iteración, se sigue cumpliendo la condición (2 es menor que 3); se imprime 2 y ahora `i` vale 3.
4. Tras eso, se revisa si se cumple la condición. Ahora, ya no se cumple: 3 no es menor que 3. El bloque de código se deja de ejecutar.

9. Ciclo for

El ciclo `for` ejecuta un bloque de código para cada elemento de un iterable. Un **iterable** es cualquier objeto que puede recorrerse elemento por elemento. Conocemos tres tipos de datos que son iterables: cadenas de caracteres, listas y diccionarios.

Un ejemplo de ciclo `for` es el siguiente:

```
for char in "Hola":
    print(char)
```

En ese ejemplo:

- El iterable es la cadena de caracteres "Hola". Los elementos que se recorren son: H, o, l, a.
- La variable `char` representa un elemento distinto en cada iteración.
- El bloque de código imprime el valor de `char` en cada iteración.

Con lo anterior en mente, resulta evidente que el ciclo se ejecuta 4 veces: en la primera iteración, `char` vale H y se imprime H; en la segunda `char` vale o y se imprime o; en la tercera, se imprime l; y en la cuarta, a.

La sintaxis general es:

```
for variable de iteración in iterable:
    bloque de código
```

10. ¿while o for?

Técnicamente, los ciclos `while` y `for` son equivalentes. Es decir, ambos pueden resolver los mismos problemas igual de bien. Sin embargo, por algo existen ambos: no tiene sentido usarlos para lo mismo.

10.1. De for a while

La conversión radica en recorrer el iterable que recorrería el ciclo `for` pero mediante un ciclo `while`. Para eso, el ciclo `while` simplemente incrementa un contador y se usa ese contador para acceder a los elementos del iterable. Es decir:

```
i = 0
while i < len(iterable):
    variable de iteración = iterable[i]
    bloque de código
    i += 1
```